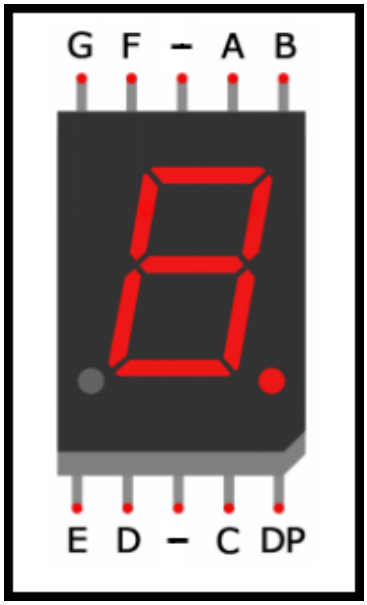
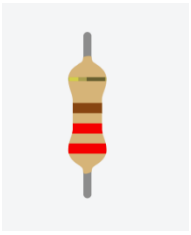
7-Segment Reaction Game

For this project, you'll create a reaction game using a new component (7-segment display). In this game, the numbers 1 through 9 will be displayed in ascending order. After reaching 9, the number will wrap around to 1 and start again.

The aim of the game is for the user to press a button while all the segments are lit up in order to earn a point.



Materials



Resistor: **1 x** 220 Ω or 330 Ω

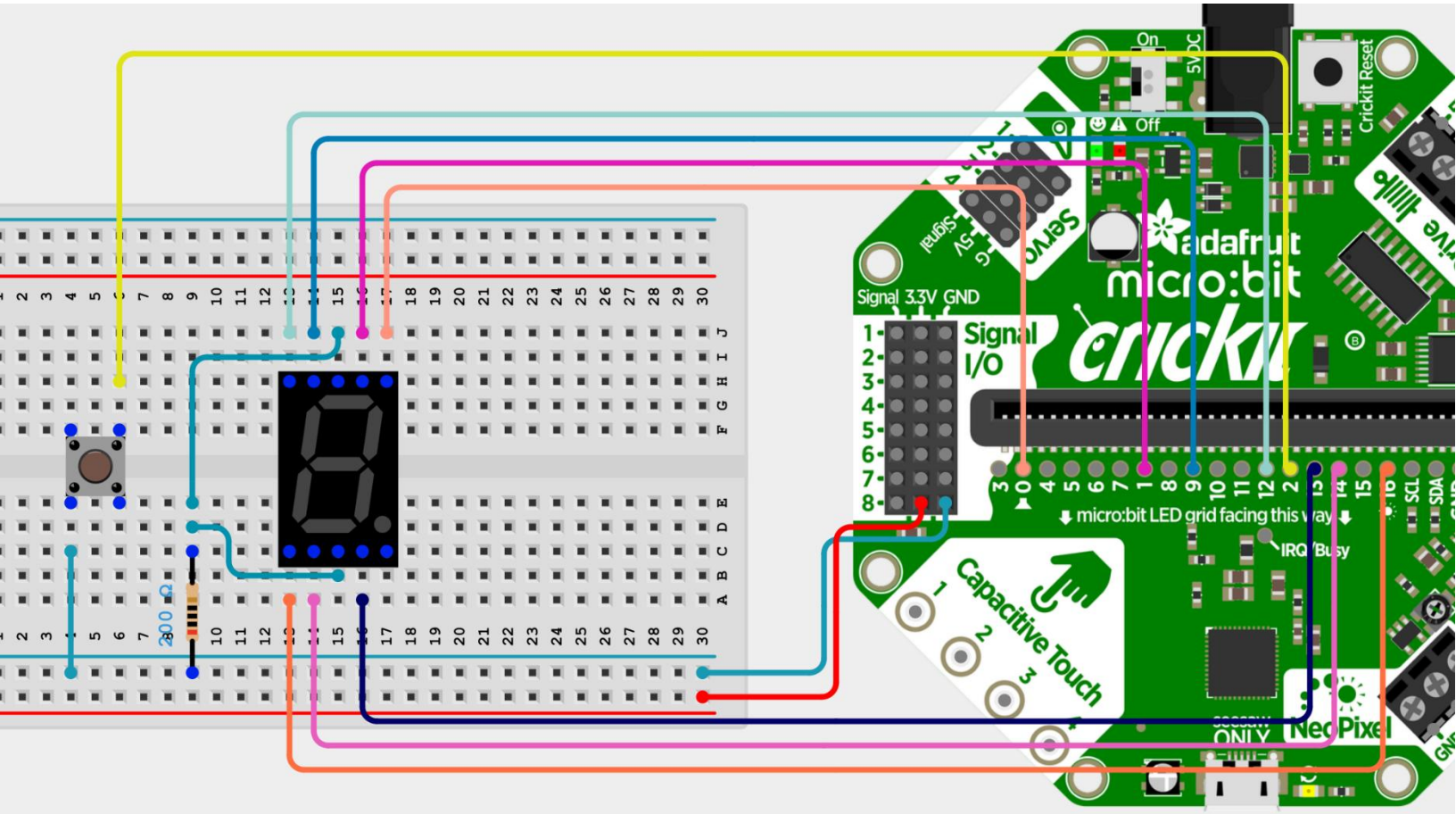
1 x 7-Segment Display *(Common Cathode)*

1 x Pushbutton

** Common Anode can be used as well, but diagram below would need to be modified so that common connection goes to 3V rather than GND*

Wiring Diagram

Wire your project according to the following schematic.

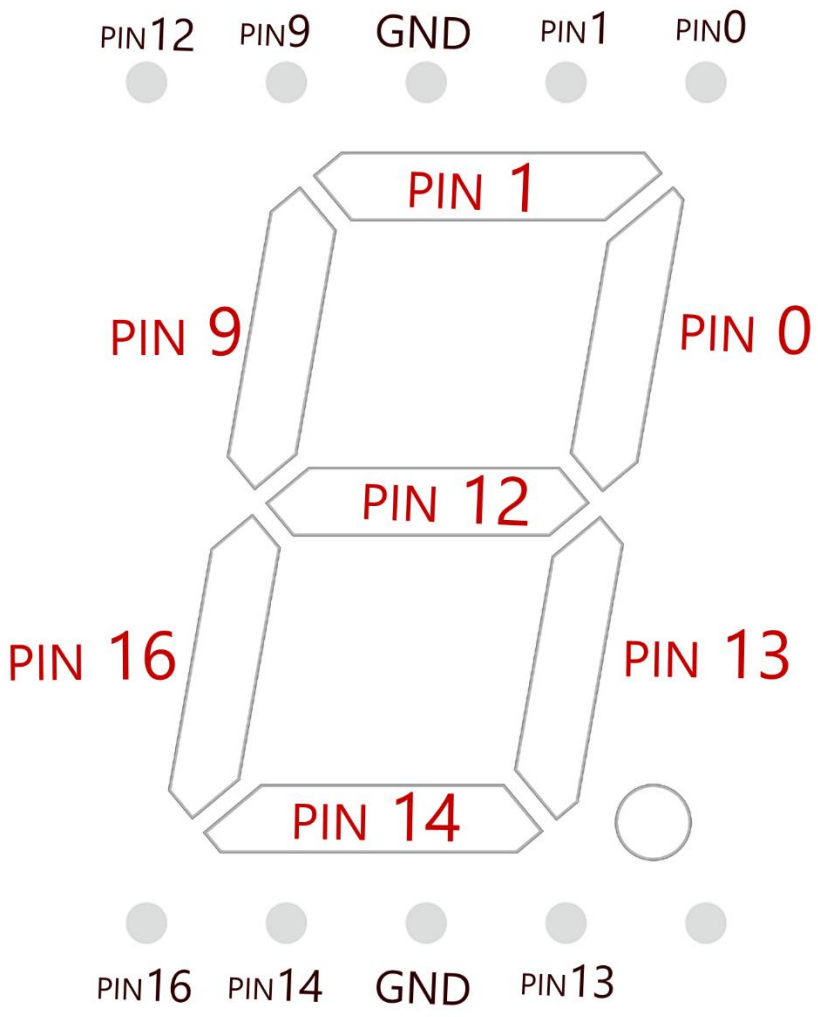


Verify Circuit

It always is a good idea to **verify that you've wired the components correctly**, by individually running a simple test on each.

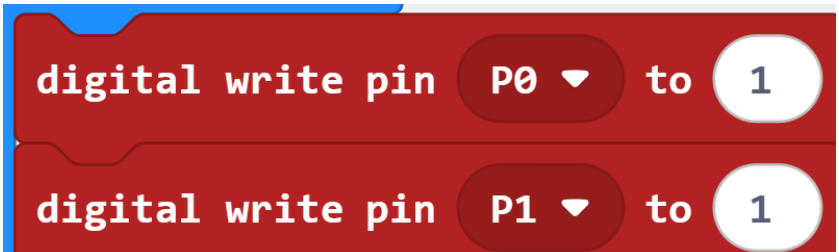
Reference: LED Pin Assignments

It can be tricky to remember which pin controls which segment. Use the following diagram, which matches the wiring details in the diagram provided previously:



Test One: 7-Segment LEDS

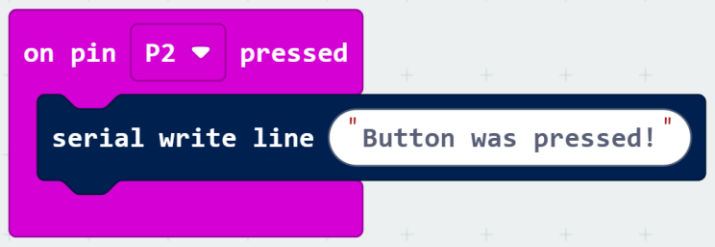
You should have wired the different segments to pins 0, 1, 9, 12, 13, 14, and 16. Set each pin to HIGH (1) and we should see all segments lit up.



Test Two: Button

We have one buttons to test, which should be connected to pin 2. This will be how the user interacts with the game.

To test you've wired the button correctly, use the **Serial Monitor** to print a simple message when a button press is detected:



Download your Test Code

Running the test program as described above should **verify the LEDs and pushbutton components are working.** If some part doesn't behave as expected here, go back to troubleshoot before moving on to the main program.

Assignment Details

For this assignment, completing the following base features will earn you a maximum mark of 10/10.

<u>Create the number functions:</u>

For this game, we'll need to be able to display the numbers 1-9 on the seven-segment display.

- Create **nine functions** (one for each of these digits) that will set the LEDs to the correct on/off pattern to represent the corresponding numbers.

<u>Start the Gameplay</u>

This reaction game will have the numbers from 1 – 9 be shown on the seven-segment display **one at a time**, in **ascending order.** Initially, each digit should be displayed for 250ms. The goal of the user will be to press the pushbutton while **lucky number 7** is visible.

- If the button is pressed at the correct time, the user should get 1 point and the message "Correct! One point." should display on the Serial Monitor.
    - The current **total score** should also be printed out.
- If the button is pressed at the incorrect time, the message "Incorrect!" should display in the Serial Monitor. *(and the pattern of numbers should continue playing)*

Winning Animation:

The game should continue until the user has 5 points:

- Print a "You Win!" message to the Serial Monitor
- Display a winning animation on the 7-Segment display
  - One LED on at a time in a "figure-8" pattern
  - Or a custom animation of your design

Increasing Difficulty

As the user scores points, the game should increase in difficulty. Have the wait time in between each displayed number depend on the user's current score.

score == 0: delay in between 250ms
score == 1: delay in between 200ms
score == 2: delay in between 150ms
score == 3: delay in between 125ms
score == 4: delay in between 100ms

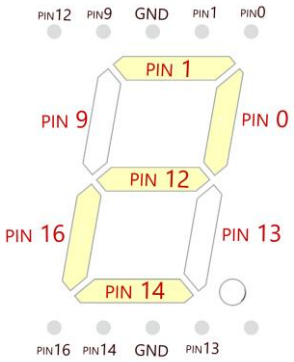
You can modify the delay lengths to increase the difficulty further if you like…

------ Suggestions and Hints are below but try to build what you can on your own first! ------

Hints and Suggestions

Create the number functions:

Each function should have 7 **digitalWrite** or **analogWrite** blocks (one per LED segment). Using the diagram in the *Reference: LED Pin Assignments* section of this document, set each LED to on/off to make a particular digit. For example:



We were to chart out the LEDs (which need to be on, which need to be off) for the number **2**, your code might look like this (above).

Use the same approach for all the other 8 number functions needed.

Start the Gameplay

In terms of **variables**, you should create one for 

as well as 

Two Forever Loops can make the control of “which number is being displayed” simple.

PSUEDO-CODE:

Forever Loop One:

Pause(ms) **delayLength**

if **currentNumber == 1:**

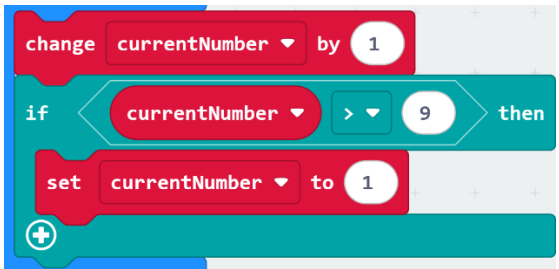
Call function **digitOne()**

Else if **currentNumber == 2:**

Call function **digitTwo()** And so on…

Forever Loop Two:

Pause(ms) **delayLength**



Winning Animation:

One way to make an animation would be to turn on the different segments *one at a time*, in some pattern.

Creating a helping function could make the code a bit simpler:

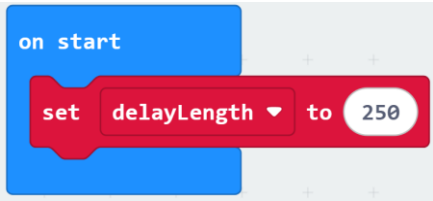
This function allows us to set all 7 segments at once.

Then, we can create a pattern by calling this function several times with different LEDs turned on (with a short delay between each):

And so on…

Increasing Difficulty

You'll need a variable to store the delay length.



Then, each time the 

value is increased, also update **delayLength.** One way would be to create a function called **updateDelay(),** with one big IF / ELSE IF / ELSE IF / etc…

If score === 0,

Then set the **delayLength** to 250ms

Else if score == 1,

Then set the **delayLength** to 200ms

And so on…